

Human-Computer Interaction Lab 3

Image Search System Report

1. Project Overview

This project implements a visual image search system for grocery product images. The system uses Gradio to build the interactive interface and uses the CLIP model to map text and images into the same vector space. A local image embedding index is then used to retrieve visually or semantically similar images. Users can search for images by entering a text description, or they can upload an image to find visually similar product images. The retrieved results are ranked by a combined similarity score. Users can also adjust the number of returned images, set a minimum score threshold, filter results by category, and use selected images by saving them to favorites or preparing them for download.

2. Dataset

The dataset used in this project is GroceryStoreDataset, available from:
<https://github.com/marcusklason/GroceryStoreDataset>

GroceryStoreDataset is a grocery product image dataset. Its content mainly covers three high-level groups: fruits, vegetables, and packaged products. In this project, the dataset is stored under `data/images/GroceryStoreDataset-master/dataset`, including training images, validation images, test images, and iconic product images. The core dataset split contains 5502 images. Since the index builder scans the whole `data/images` directory, which also includes extra sample images, the current local retrieval index contains 5502 images in total. The image embedding dimension is 512.

The dataset scale used in this project is shown below:

Split	Number of Images
train	2640
val	296
test	2485
iconic-images-and-descriptions	81
Total	5502

The dataset contains 81 fine-grained classes and 43 coarse-grained classes. These classes are mainly grouped into Fruit, Vegetables, and Packages. Typical categories include Banana, Kiwi, Apple, Orange, Tomato, Zucchini, Milk, Juice, and Yoghurt. Since grocery products have clear category names and a hierarchical directory structure, this project uses not only CLIP vector similarity for ranking, but also lightweight category-name boosting based on image paths. This improves retrieval quality for short product queries such as kiwi, banana, and orange juice.

3. System Implementation

The system mainly consists of three code files:

- (1) `build_index.py`: reads dataset images, uses the CLIP image encoder to generate image features, and saves the local index.
- (2) `search.py`: handles image scanning, CLIP model loading, text and image encoding, similarity calculation, and ranking logic.
- (3) `app.py`: builds the Gradio visual interface and connects user actions to search, filtering, saving, and downloading functions.

During index construction, the system recursively scans image files under `data/images`. Each image is passed into the CLIP image encoder to obtain a 512-dimensional image vector. All vectors are L2-normalized and saved as `index/image_embeddings.npy`. The corresponding image paths are saved as `index/image_paths.json`, and metadata is saved as `index/metadata.json`. With this design, the interface does not need to recompute all dataset image features at runtime. It only needs to load the prebuilt local index.

In text search mode, after the user enters a text query, the system expands the original query into several prompt variants, such as the original query, a photo of ..., a grocery store product photo of ..., and fresh These texts are encoded by the CLIP text encoder. Their vectors are averaged and normalized as the final query vector. The system then calculates dot-product similarity between the query vector and all image vectors, and returns results sorted by score in descending order. To make short product-name queries more stable, the system also applies a lightweight boost using category information extracted from image paths.

In image search mode, the user uploads an image. The system uses the CLIP image encoder to generate a query image vector and compares it with image vectors in the index. If the user also enters a text hint, the system combines the image vector and text vector with predefined weights, producing a query vector that uses both visual information and semantic hints. If no text hint is given, the system infers a possible

category hint from the nearest neighbor and applies a weaker category boost to assist ranking.

4. Interface Design and the Five-Stage Search Framework

The interface design of this project reflects the Five-Stage Search Framework: Formulation, Initiation of Action, Review of Results, Refinement, and Use.

4.1 Formulation: Expressing the Search Need

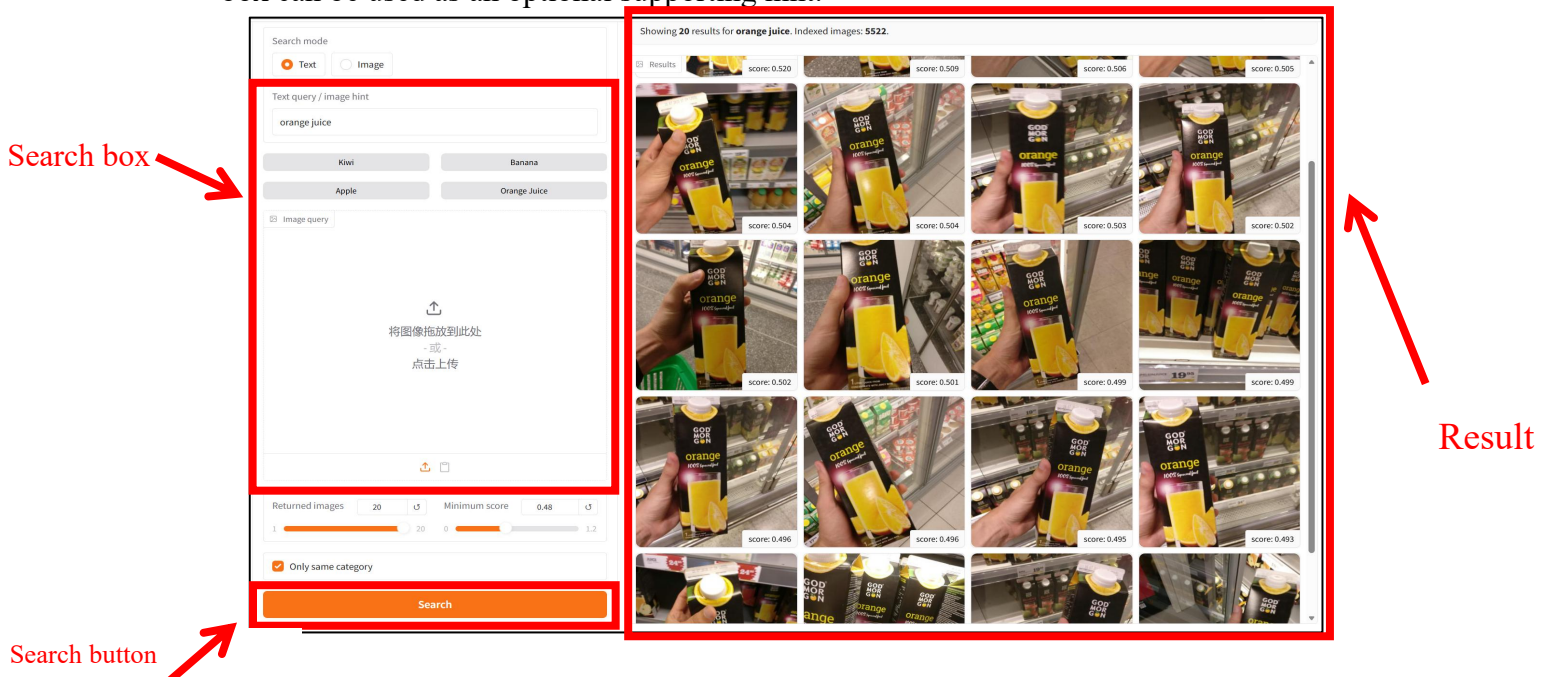
In the formulation stage, the interface provides two input methods: a text input box and an image upload component. Users can directly enter a product name or description, or upload an existing product image as the query. To reduce input effort, the interface provides quick query buttons such as Kiwi, Banana, Apple, and Orange Juice, allowing users to fill in common queries with one click.

For image queries, the upload component directly displays the selected query image. Users can confirm whether the image is correct before starting the search, which reduces the possibility of uploading the wrong image or performing an incorrect search.

4.2 Initiation of Action: Starting the Search

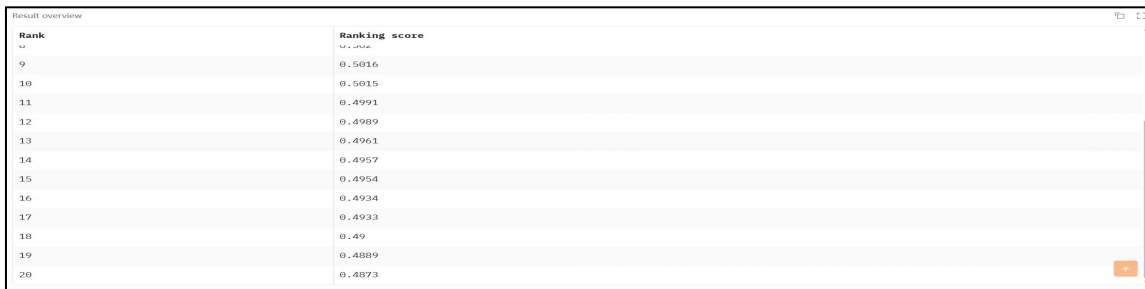
The interface uses an explicit Search button to start retrieval. Users first complete the query content, search mode, and parameter settings, and then click the button to initiate the search. This explicit action is suitable for this project because CLIP inference may take some time. It allows users to confirm their input before execution and avoids expensive computation being triggered by every small input change.

The search mode is selected with radio buttons between Text and Image. This design makes it clear how the system will interpret the input. In Text mode, the text box is the main query. In Image mode, the uploaded image is the main query, while the text box can be used as an optional supporting hint.



4.3 Review of Results: Viewing and Understanding Results

The search results are displayed in an image gallery. Each result includes an image preview and a corresponding ranking score. The overview area at the top of the interface shows the current number of returned results, the query content, and the total number of indexed images. This helps users quickly judge whether the search was successful and understand the scale of the image collection being searched.



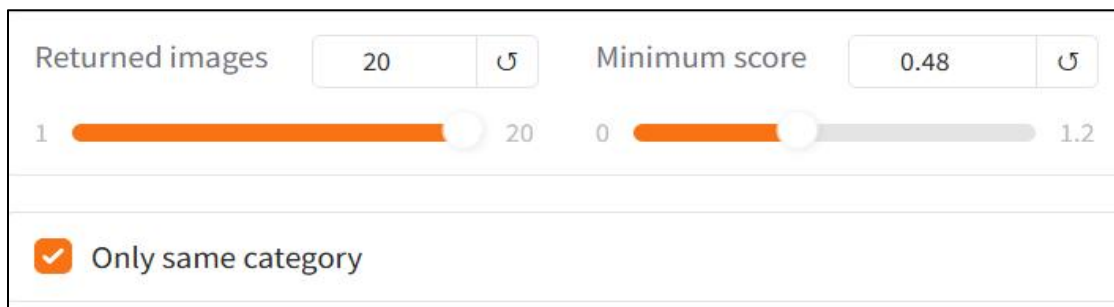
Rank	Ranking score
9	0.5016
10	0.5015
11	0.4991
12	0.4989
13	0.4961
14	0.4957
15	0.4954
16	0.4934
17	0.4933
18	0.49
19	0.4889
20	0.4873

The interface also provides a result table listing each result's rank and score. Users can quickly browse image content through the gallery and compare ranking differences through the table. A dropdown component below the result area allows users to select a specific result, and the selected image is previewed on the right side for closer examination.

4.4 Refinement: Adjusting and Improving the Search

When users are not satisfied with the results, the interface allows them to adjust several search parameters directly. The Returned images slider controls the number of returned images, from 1 to 20. The Minimum score slider filters out low-score results. The Only same category checkbox keeps only images from the same category as the top-ranked result. These parameters help users narrow the result set when there are too many results, when the results are too mixed, or when users want to focus on similar products.

Users can also modify the text query, switch search mode, upload another image, or add a text hint in image search mode at any time. When no results are found, the system provides a meaningful message, such as suggesting that the user lower the minimum score. This supports progressive refinement: users do not need to move to another page, but can continuously adjust search conditions and search again within the same interface.



Returned images: 20 (slider from 1 to 20)

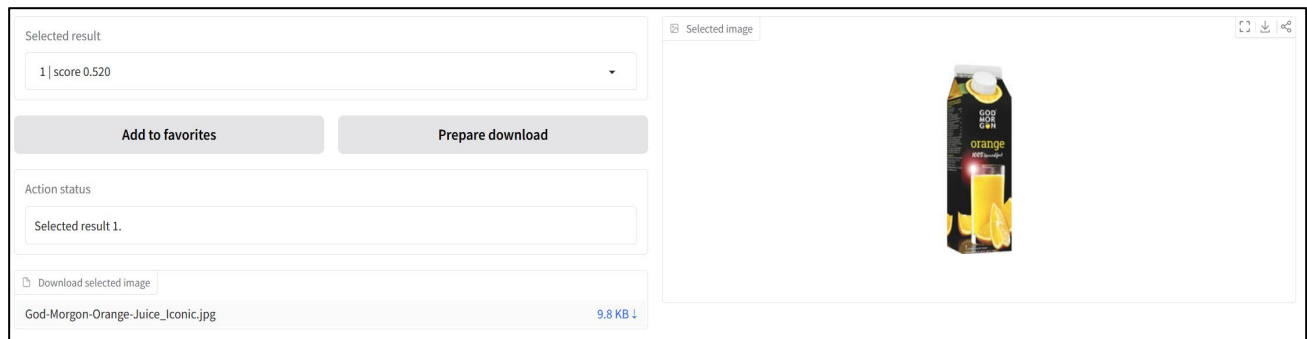
Minimum score: 0.48 (slider from 0 to 1.2)

☒ Only same category

4.5 Use: Using Retrieved Results

The final goal of search is not only to view results, but also to use them. Therefore, the system provides two actions for selected images: Add to favorites and Prepare download. Users can copy an interesting image to the local favorites/ folder, or prepare the selected image for download. This corresponds to the Use stage in the Five-Stage Search Framework and satisfies the assignment requirement that users should be able to use retrieved images, such as by downloading or saving them.

The favorites function adds a timestamp to the saved filename, preventing files from different searches from overwriting each other. The download function passes the selected result to Gradio's file component, allowing users to save or use the selected image further.



5. Influence of Text and Image Input on User Workflow

Although both text search and image search are eventually converted into CLIP vector retrieval, they affect the user workflow in different ways.

Text input is fast and direct. It is suitable when users already know the target product name or keyword. For example, if a user wants to find bananas, kiwis, or orange juice, entering the product name is enough to start the search. However, text input also requires users to know or guess appropriate words. If the dataset category names are different from the user's natural expression, or if the user only remembers the appearance but not the product name, text search may feel less natural.

Image input is more suitable when users have an example image but cannot easily describe the product in words. Users can upload a similar image, and the system retrieves results based on visual features such as color, shape, and package appearance. Image input reduces the burden of verbal expression, but it also increases operation cost. Users need to prepare an image, upload it, and confirm that the uploaded content is correct. In addition, if the query image has a complex background or a very different shooting angle, the retrieved results may be affected by visual noise.

To make both input methods user-friendly, this project uses the following design choices:

- (1) Both modes share the same result display area and result operation workflow. Whether users search with text or with an image, they see the same gallery, scores, result table, favorites function, and download function.
- (2) Image mode keeps the text box as an optional hint. After uploading an image, users can enter hints such as kiwi or apple, helping the system balance visual similarity and semantic category information.
- (3) Text mode provides quick query buttons, reducing input effort for first-time users and helping users quickly understand what kinds of products the system can search for.
- (4) The upload component provides a query image preview, reducing mistakes in image search mode.
- (5) Parameter controls are available for both modes, including the number of returned images, minimum score, and same-category filtering. This allows both types of users to continue refining their searches.

Therefore, this project does not treat text search and image search as two isolated functions. Instead, they share the same process for searching, browsing, filtering, and using results. This lowers the learning cost and follows interaction principles such as consistency and recoverability in search interface design.

6. How to Run

- (1) install the dependencies: `pip install -r requirements.txt`
- (2) If the index has not been built, run: `python build_index.py`
- (3) After the index is built, run the Gradio interface: `python app.py`
- (4) After the program starts, open the local address shown in the terminal. It is usually: `http://127.0.0.1:7860`

7. Summary

In summary, this project builds a CLIP-based image search system with a Gradio interface. It supports both text and image queries, ranked result review, parameter refinement, and result use through favorites or download. The interface follows the Five-Stage Search Framework and keeps the two query modes consistent and user-friendly.